Rede Neural Artificial

Explicação, Características e Tipos

Baseado na *Lecture 4 — Neural Networks and Backpropagation*
CS231n, Stanford (Fei-Fei Li, Yunzhu Li, Ruohan Gao, 2023)

José Eduardo Araujo Costa Melo

1 Introdução

Os slides da Lecture 4 apresentam a **rede neural artificial** como uma evolução natural do classificador linear. Enquanto o classificador linear aplica apenas uma transformação $f = Wx$, a rede neural empilha **múltiplas transformações lineares intercaladas com funções de ativação não-lineares**, o que lhe confere muito mais poder de representação.

A rede neural é apresentada como o bloco fundamental do *deep learning* — a mesma estrutura que, quando empilhada em muitas camadas e combinada com operações especializadas (como convoluções), dá origem a modelos modernos como AlexNet, GPT-4 e SAM.

2 Definição

Uma rede neural *fully-connected* (também chamada de **MLP — Multi-Layer Perceptron**) é uma função composta da forma:

$$f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$

Para uma rede de 3 camadas:

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

Onde:

- $x \in \mathbb{R}^D$ é o vetor de entrada;
- $W_1 \in \mathbb{R}^{H \times D}$, $W_2 \in \mathbb{R}^{C \times H}$, etc. são as matrizes de pesos;
- $\max(0, \cdot)$ é a **função de ativação** (ReLU no exemplo);
- Na prática, adiciona-se um **bias** aprendível em cada camada.

Observação terminológica (slide 21): "Neural Network" é um termo muito amplo. O nome mais preciso para essa arquitetura é *fully-connected network* ou *multi-layer perceptron*.

Modelo	Função
Classificador linear	$f = Wx$
Rede de 2 camadas	$f = W_2 \max(0, W_1 x)$
Rede de 3 camadas	$f = W_3 \max(0, W_2 \max(0, W_1 x))$

3 Como a rede neural funciona

3.1 Do classificador linear à rede

3.2 O papel da não-linearidade

Esse é o ponto central dos slides (19–20, 25–26). Sem a ativação, a composição de camadas colapsa:

$$f = W_2 W_1 x = W_3 x \quad \Rightarrow \quad \text{volta a ser um classificador linear}$$

A não-linearidade permite que a rede **aprenda transformações de features** que tornam os dados separáveis. O exemplo clássico (slide 20) é o dos pontos vermelhos no centro e azuis em volta: nenhuma reta os separa, mas após uma transformação (ex.: coordenadas polares r, θ) eles se tornam linearmente separáveis. A rede aprende essa transformação automaticamente.

3.3 Computação hierárquica (slide 23)

Cada camada transforma a representação:

$$x\ (3072)\ \rightarrow\ h\ (100)\ \rightarrow\ s\ (10)$$

No exemplo do slide, uma imagem CIFAR ($32 \times 32 \times 3 = 3072$ dimensões) é projetada em 100 neurônios ocultos e depois em 10 scores de classe.

3.4 Aprendizado de templates (slide 24)

- O classificador linear aprende **1 template por classe** (10 templates para 10 classes).
- A rede de 2 camadas aprende **100 templates** na camada oculta e os **combina** na saída.
- Os templates são **compartilhados** entre classes → representação mais rica e eficiente.

3.5 Forward pass e backward pass

O treinamento segue o ciclo:

1. **Forward pass:** calcula a saída e a loss.
2. **Backward pass (backpropagation):** aplica a regra da cadeia para calcular os gradientes.

3. **Atualização:** gradient descent atualiza os pesos.

Exemplo do slide 30 (rede de 2 camadas em ∼20 linhas de NumPy):

```
h = 1 / (1 + np.exp(-x.dot(w1)))            # ativacao sigmoid
y_pred = h.dot(w2)                          # saida
loss = np.square(y_pred - y).sum()          # MSE

grad_y_pred = 2.0 * (y_pred - y)
grad_w2 = h.T.dot(grad_y_pred)
grad_h = grad_y_pred.dot(w2.T)
grad_w1 = x.T.dot(grad_h * h * (1 - h))     # derivada da sigmoid

w1 -= 1e-4 * grad_w1
w2 -= 1e-4 * grad_w2
```

4 Características da rede neural

4.1 Características estruturais

Característica	**Descrição**
Camadas empilhadas	Entrada → camadas ocultas → saída
Fully-connected	Cada neurônio de uma camada se conecta a todos da anterior
Não-linearidade	Ativação entre camadas (essencial)
Bias aprendível	Um termo aditivo por camada (na prática)
Parâmetros	Matrizes de pesos $W_1, W_2, \ldots$ treináveis

4.2 Características de representação

- **Maior poder representacional** que classificadores lineares.
- **Aprende features hierárquicas:** camadas iniciais capturam padrões simples; camadas finais, combinações complexas.
- **Compartilhamento de templates** entre classes.
- **Capacidade controlada pelo número de neurônios:** mais neurônios = mais capacidade (slide 35).

4.3 Características de treinamento

- **Loss total** = perda de dados + regularização:

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i + \lambda R(W_1) + \lambda R(W_2)$$

- **Otimização por gradient descent** (SGD, Momentum, RMSProp, Adam).
- **Gradientes via backpropagation** (regra da cadeia no grafo computacional).
- **Regularização** deve ser usada em vez de reduzir o tamanho da rede (slide 36): redes grandes + λ forte generalizam melhor que redes pequenas.

4.4 Funções de ativação (slide 27)

Ativação	Fórmula	Observação
Sigmoid	$\sigma(x) = \dfrac{1}{1+e^{-x}}$	Satura em 0 e 1
Tanh	$\tanh(x)$	Centrada em zero
ReLU	$\max(0, x)$	**Padrão recomendado**
Leaky ReLU	$\max(0.1x, x)$	Evita neurônios mortos
Maxout	$\max(w_1^T x + b_1,\ w_2^T x + b_2)$	Generaliza ReLU
ELU	x se $x \geq 0$; $\quad \alpha(e^x - 1)$ se $x < 0$	Suave para negativos

4.5 Analogia biológica (slides 37–44)

O neurônio artificial é inspirado no biológico:

- Entradas $\leftrightarrow$ dendritos;
- Soma ponderada + bias $\leftrightarrow$ corpo celular;
- Ativação $f \leftrightarrow$ disparo do axônio.

Atenção: *"Be very careful with your brain analogies!"* Dendritos fazem computações não-lineares complexas; sinapses não são um único peso. A analogia é inspiração, não equivalência.

5 Tipos de rede neural apresentados

5.1 Quanto ao número de camadas

Tipo	Fórmula	Nomenclatura
Classificador linear	$f = Wx$	0 camadas ocultas
Rede de 2 camadas	$f = W_2 \max(0, W_1 x)$	1 camada oculta
Rede de 3 camadas	$f = W_3 \max(0, W_2 \max(0, W_1 x))$	2 camadas ocultas
Rede profunda	$f = W_L \max(0, \ldots \max(0, W_1 x))$	Muitas camadas ocultas

Atenção: A camada de entrada **não conta** na nomenclatura. "2-layer Net" = 1 camada oculta.

5.2 Quanto ao tipo de conexão

- **Fully-connected (densa):** todos os neurônios conectados — foco da Lecture 4.
- **Redes com conexões aleatórias:** também funcionam (Xie et al., 2019, slide 43).
- **Redes convolucionais (CNNs):** próxima aula — conexões locais e compartilhamento de pesos.

5.3 Quanto à aplicação (exemplos dos slides iniciais)

- **Geração texto→imagem** (Ramesh et al., 2022, slide 13).
- **Modelos multimodais** (GPT-4, slide 14).
- **Segmentação de imagens** (SAM, slide 15).
- **Classificação de imagens** (AlexNet, slide 49).

6 Arquitetura visual (resumo)

$$x \xrightarrow{W_1} h_1 \xrightarrow{W_2} h_2 \xrightarrow{\cdots} s$$

Fluxo:

- **Forward:** $x \to h_1 \to h_2 \to \cdots \to s$
- **Backward:** $\dfrac{\partial L}{\partial s} \to \dfrac{\partial L}{\partial h_2} \to \dfrac{\partial L}{\partial h_1} \to \dfrac{\partial L}{\partial W_1}$

7 Resumo final (slide 145)

- Redes fully-connected são **pilhas de funções lineares + ativações não-lineares**; têm muito mais poder representacional que classificadores lineares.
- **Backpropagation** = aplicação recursiva da regra da cadeia ao longo de um grafo computacional.
- Implementações mantêm um grafo onde cada nó implementa `forward()` / `backward()`.
- **Forward:** computa o resultado e salva intermediários.
- **Backward:** aplica a regra da cadeia para obter gradientes.

8 Conclusão

A rede neural apresentada por Fei-Fei Li é uma **composição de transformações lineares e não-linearidades**, treinada por gradient descent com gradientes obtidos por backpropagation. Suas características centrais são:

1. **Empilhamento de camadas** com ativações não-lineares;
2. **Aprendizado de features hierárquicas** e templates compartilhados;
3. **Capacidade controlada** pelo número de neurônios;
4. **Regularização** como forma de generalização (não reduzir a rede);
5. **Backpropagation** como mecanismo universal de cálculo de gradientes.

É a base sobre a qual se constroem as arquiteturas modernas de deep learning.